Coursework Specifications for CGR 2026

Released: 5th Oct	Due: 11th Dec 2026	Marks: 100	Contribution to final grade: 80%

1. GENERAL GUIDANCE 1
   1.1 Overview 1
   1.2 Coding requirements 1
   1.3 Use of AI (encouraged for coding but not for writing reports) 2
   1.4 Submission 2
   1.5 Intermediate Checkpoints and Timeliness Bonus 4
   1.6 Final Phase 4
2. MODULE DETAILS 4
   2.1 Module 1: Core Renderers 4
   2.2 Module 2: Deeper Analysis of Renderer Components 6
   2.3 Module 3: Acceleration and Testing 7
   2.4 Final Phase 8
3. REPORT STRUCTURE 8
   3.1 General Guidance on Writing Reports 8
   3.2 Module Report Structure 9
   3.3 Final Report Structure 9
4. Marking scheme 10
5. Exceptionalism 11
6. Getting started 11

1. GENERAL GUIDANCE

1.1 Overview

This coursework requires you to implement a physically based renderer in C++ from scratch. The renderer must support two ray-tracing types, a Whitted-style ray tracer (WSRT) and a distributed ray tracer (DRT), and must be extended across three modules, each submitted at an intermediate checkpoint. The final submission consolidates all work and incorporates advanced features. The coursework is designed to develop both implementation skill and critical understanding of rendering algorithms. You are expected to demonstrate each feature through carefully designed test scenes, quantitative measurements, and written analysis.

1.2 Coding requirements

All code must be written in C++ using only the C++ Standard Library. No external libraries, rendering frameworks, linear algebra packages, parsing toolkits or image-loading libraries are permitted. Any

mathematical utilities such as vector arithmetic, matrix operations, and image output must be implemented. This constraint is intentional and should be practical, since you are allowed to use generative AI to produce code. This coursework is meant to be solved individually by students. Active discussion between students is strongly encouraged. Since many of the marked elements involve the creative design of scenes and test cases along with explanation and justification pertaining specifically to those scenes, it is expected that there will be minimal overlap in terms of the submitted reports.

1.3 Use of AI (encouraged for coding but not for writing reports)

Coding: The use of Artificial Intelligence tools is **allowed and encouraged** (ELM's Claude Sonnet 4.6 is recommended) for coding tasks. It is mandatory to acknowledge the extent of reliance on these tools, in a section of the report. Failure to do so would constitute academic misconduct. For example, if it generated all the code for the WSRT then provide an example of the prompt that was used. If the code had errors, that needed iterative fixing, provide some examples of the errors and how involved the fixes were. If the code compiled but executed with runtime errors, explain how those errors were detected and fixed. If agentic workflows are used, then mention explicitly the extent of human (i.e. your) input and post-processing.

Writing the report: AI tools are **not allowed** for report writing. Not even for checking spelling or grammar---errors in spelling or English grammar will not be penalised. Please keep your report brief. They should essentially show the required images along with the requested explanation, for each subsection. The use of AI for generating the report or even sections of it will be regarded as academic misconduct. If you use AI for researching, please do not use its answers (or portions of them) verbatim.

1.4 Submission

1.4.1 Folder Structure

Your submission must be organised as a single top-level folder named with your exam number, for example B012345 (submit as B012345.zip), contain the following subfolders and files exactly as described below. Submissions that do not conform to this structure may incur a mark penalty.
The top-level folder must contain the following:

```
<exam_number>/
├── featureList.txt
├── renderReportedImages.(sh|bat|py)
├── README
├── source/
├── scenes/
├── reports/
|   ├── Module1.pdf
|   ├── Module2.pdf
|   ├── Module3.pdf
|   └── <exam_number>.pdf
├── output/
└── studentTests/
    ├── runAllTests.(sh|bat|py)
    ├── test1/
    ├── test2/
    └── ...
```

featureList.txt	a plain-text file listing every implemented feature, one per line. Features should correspond directly to the tasks described in the specifications. Partially implemented features should be listed with a brief qualifier, for example: Halton sampler implemented but not integrated into area light sampling. This file is used by markers to calibrate expectations before reading the report and inspecting code.
renderReportedImages	a script that first builds the project and then renders all the raytraced images shown in the report, into the output folder.
README	a plain-text or Markdown file containing step-by-step instructions for building the project including compiler version, build system, and any flags required. It must also state the operating system and hardware environment in which the code was developed and tested, for example macOS 14.4, Apple M2, clang 15.0. It must include instructions for running the renderer on a given .pbrt scene file, instructions for running all student test cases via the top-level test script, and any known limitations or platform-specific considerations.
source/	all C++ source files with .cpp, .h, and .hpp extensions. Code must compile cleanly from this directory using the instructions in the README. Do not include compiled binaries or object files.
scenes/	one .pbrt scene file for every scene rendered and discussed in any of the three module reports or the final report. Scene files must be complete and self-contained. Any geometry or texture data referenced by a scene file must also be present within this directory, in subdirectories if appropriate. A marker must be able to reproduce every image in the reports by running the renderer on the corresponding scene file.
reports/	PDF reports for each module and the consolidated final report. Report content and structure are specified in Section 3.
output/	may be submitted empty. However, executing the renderer as described in the README must write all output images to this folder. Output images should be in a widely readable format; PPM or PNG is recommended, as both are implementable without external libraries. Log files with a .log extension must also be written to this folder. Your renderer must produce the executable 'CGR26_renderer' within this folder when built.
studentTests/	should contain a top-level script named runAllTests that executes every test case in sequence and generates a log. It must also contain numbered subdirectories named test1, test2, and so on. Each subdirectory must contain everything needed for that test: a scene file, any referenced geometry or texture data, an execution script or clear instructions, and a reference image or expected output value where applicable. The test suite will be shared with the rest of the cohort and must therefore be self-contained and documented clearly enough for another student to run it without modification on their own renderer.

1.4.2 Final Submission Format

Submit the top-level folder as a compressed ZIP archive exam_number.zip and submitted via Learn. Do not include hidden files, .DS_Store files, build artefacts, or large binary assets unrelated to the project.

1.5 Intermediate Checkpoints and Timeliness Bonus

There are three intermediate checkpoint deadlines, one per module. At each checkpoint you must submit the module report as a PDF together with any code written until then. The submission structure should be the for checkpoints as described above. The checkpoint provides you with an opportunity to identify hurdles and seek assistance, early enough in the development cycle and to encourage steady progress. The final submission must include sections on the three modules. These may be submitted verbatim from the checkpoint-submitted versions, or with revisions. Timeliness bonus marks are awarded based on two criteria:

- The overlap between the checkpoint and the final version. A final report that is essentially unchanged from the checkpoint, indicating that the work was completed on time and required no major revision, will receive maximum timeliness bonus for that module.
- Justified improvements. If you identify a genuine improvement to an earlier implementation after the checkpoint, for example a more efficient BVH construction algorithm or a corrected BRDF sampling distribution, you may revise the relevant section and explain the change clearly in the final report. Quantitative evidence comparing the checkpoint and final versions, such as runtime measurements, error plots, or side-by-side renders, will be used to assess whether the revision is well-motivated.

Timeliness bonus marks are awarded subjectively at the marker's discretion. Claims of improvement that are unsubstantiated, or that represent corrections to incorrect implementations rather than genuine optimisations, will receive reduced or no bonus.

1.6 Final Phase

Following submission of all three modules, the coursework enters a final phase comprising two additional deliverables.

- The first is an advanced implementation covering importance sampling. You will extend the DRT to support importance sampling of material BRDFs and demonstrate quantitatively that this reduces variance compared to uniform hemisphere sampling.
- The second is a rigorous testing phase using a pooled test suite. Test cases submitted by all students will be pooled and redistributed. You will run the pooled test suite against your final renderer and include a summary of results in the final report, covering pass rates, identified failure modes, and discussion. Additional test cases may be supplied by the course organiser.

2. MODULE DETAILS

2.1 Module 1: Core Renderers

This module requires you to implement the two ray-tracing approaches. Both must read scene descriptions written in a subset of PBRT's input file format. The integrator tag in the scene file determines which approach (‘whitted’ or ‘distributed’) will be used by your executable renderer .
Both integrators must write output images to the output/ directory. Both must write a .log file alongside each output image, recording any warnings or errors encountered during parsing or rendering. If the input file contains a syntax error or an unsupported feature, this must be logged with a descriptive message rather than causing a silent failure or crash. Both integrators must support code profiling to report per-frame render time and optionally per-stage timings such as parsing, BVH construction, and shading. These timing reports should be written to the log file. Where you compare your output to PBRT, document the scene file used and the specific pixels or image statistics compared.

2.1.1 Whitted-Style Ray Tracer (WSRT)

The WSRT must implement the following features.

- *Specular reflection*: Ideal mirror reflection using the standard reflected ray direction formula. The reflection must be computed recursively up to the maximum ray-tree depth.
- *Specular refraction*: Transmission through dielectric materials using Snell's law and the Fresnel equations. Handle total internal reflection correctly.
- *Blinn-Phong shading*: Evaluate the Blinn-Phong illumination model at each intersection, including ambient, diffuse, and specular components. The model parameters, including diffuse colour, specular colour, and shininess exponent, must be configurable per material in the scene file.
- *Point light sources*: Support for point lights, with configurable position and radiant intensity.
- *Shadows*: Cast shadow rays from each intersection point towards each light source. Correctly handle self-shadowing with an appropriate ray offset.
- *Textures*: Support UV-mapped textures for diffuse colour on at least spheres and planes. Texture data may be stored as simple in-memory arrays loaded from a plain PPM file, which is implementable without external libraries.
- *Geometry*: Support planes, spheres, and individual triangles as primitive types. Each primitive must support ray intersection and UV coordinate generation for texturing.

Output images WSRT_simple and WSRT_creative. The former showing the above features, and the latter showing a creative (non-trivial) scene. Point to the specific regions (e.g. with arrows or zoomed insets) in the image that show the features. For the creative scene, use a photograph (you may set this up on your desk for example) of a real scene to inspire your creative scene. Show the photograph and the rendered image (they do not need to match) in the report.

2.1.2 Distributed Ray Tracer (DRT)

The DRT must implement the following features, building on the foundations of the WSRT.

- *Triangle meshes*: Load and render triangle meshes, for example from OBJ files parsed using only the standard library. Per-vertex normals and UV coordinates must be supported for smooth shading and texturing.
- *Textures*: As in the WSRT, textures must be applicable to mesh surfaces using interpolated UV coordinates.
- *Area light sources and soft shadows*: Model light sources as finite geometric areas such as emissive rectangles or discs. Shadow rays are cast towards randomly sampled points on the light surface, producing soft shadows. The density of sampling must be configurable.
- *Phong BRDF*: Evaluate the Phong BRDF correctly, including the cosine-weighted diffuse term and the specular lobe. Ensure energy conservation.
- *Defocus blur and depth of field*: Simulate a thin-lens camera model. Rays are jittered at the lens aperture and converge at the focal plane. Aperture radius and focal distance must be configurable in the scene file.
- *Reinhard tone mapping:* Apply Reinhard tone mapping to the HDR output of the DRT before writing the image. The mapping must be applied per-channel or to luminance, and you must document your choice. The key parameter such as the white-point must be configurable.

Output images DRT_simple and DRT_creative. The former showing the above features, and the latter showing a creative (non-trivial) scene. Point to the specific regions (e.g. with arrows or zoomed insets) in the image that show the features. For the creative scene, use a photograph (you may set this up on your desk for example) of a real scene to inspire your creative scene. Show the photograph and the rendered image (they do not need to match) in the report.

2.2 Module 2: Deeper Analysis of Renderer Components

This module requires analytical investigation and empirical comparison of specific renderer components. For each task you must design dedicated scenes, produce rendered images, provide quantitative measurements where specified, and discuss your findings.

2.2.1 Camera Models

- *Orthographic vs perspective projection*: Derive and explain the mathematical difference between the two projection models. Show the effect on rendered images using scenes where the distinction is clearly visible. Design a scene for which orthographic projection is the more appropriate choice and one scene for which perspective projection is preferable. Justify your choices using rendered views from these scenes.
- *Field of view*: Demonstrate the effect of varying the (horizontal or vertical) field-of-view angle in perspective projection. Render images of a scene with three distinct field-of-view values. Using these, explain the relationship between field of view and the apparent size and the types of distortion of objects specific to the images you show. Use arrows or zoomed insets to focus on those regions.

Output images camera_ortho, camera_persp, camera_fov1, camera_fov2, camera_fov3.

2.2.2 Antialiasing

- *Demonstrating aliasing*: Produce renders from both the WSRT and DRT that exhibit visible aliasing artefacts such as jagged edges, Moire patterns, or sharp boundary stepping. Choose scenes and camera configurations that make the aliasing obvious.
- *Supersampling antialiasing*: Implement antialiasing by casting multiple rays per pixel and averaging the results. Show renders with 1, 4, 9, and 16 samples per pixel or equivalent. Discuss the trade-off between image quality and render time.
- *Performance and alternatives*: Quantify the render time increase as a function of samples per pixel. Suggest and briefly describe at least one technique, for example adaptive sampling or post-process filtering, that could reduce this cost. You are not required to implement the alternative.

Output images aa_wsrt, aa_drt, aa_ss1, aa_ss4, aa_ss9, aa_ss16. Ensure that the scene that is used to demonstrate this is non-trivial. e.g. use multiple geometries with thin features and textures.

2.2.3 Sampler Comparison

Within the DRT, compare the following sampling strategies for soft shadows produced by area lights.

- *Uniform random sampling*: Samples drawn independently and uniformly at random over the light surface.
- *Regular grid sampling*: The light surface is divided into an n by n grid and one sample is placed at the centre of each cell.
- *Halton sequence sampling*: Low-discrepancy samples generated using the Halton sequence in two dimensions.

For each strategy, produce renders of the same scene at the same total sample count. Quantitatively compare the three strategies using a metric of your choice, for example RMSE (or variance) against a high-sample-count PBRT reference image measured at specific pixels. Discuss the conditions under which each strategy is preferable.

Output images sampler_unif, sampler_grid, sampler_halton. Use suffixes as the number of samples used. Point to specific regions in the image that are discussed (using arrows or insets)

2.2.4 DRT Parameter Study

The distributed ray tracer is governed by two principal parameters: the branching factor (BF), which is the number of rays cast at each scattering event, and the maximum depth (MD), which is the maximum number of recursive ray-tree levels. Explain how you chose appropriate values for BF and MD for a given scene, and what the cost of increasing each parameter is. Design four scenes, one well-suited to each of the following parameter combinations, and show the resulting renders with discussion.

- *Small BF, low MD*: Few samples, shallow recursion.
- *Small BF, high MD*: Few samples, deep recursion.
- *Large BF, low MD*: Many samples, shallow recursion.
- *Large BF, high MD*: Many samples, deep recursion.

The scenes should be designed so that the rendered output meaningfully reflects the parameter setting. For example, a scene with many inter-reflections is well-suited to exploring the effect of MD, while a scene with large softly lit surfaces benefits from a large BF.

Output images param_small_low, param_small_high, param_large_low, param_large_high. Use non-trivial geometries in the scene and point to specific regions in the image that are discussed (using arrows or zoomed insets).

2.3 Module 3: Acceleration and Testing

2.3.1 Acceleration Data Structure

Implement a spatial acceleration data structure to reduce the number of ray-primitive intersection tests for scenes containing large numbers of spheres or triangle mesh faces. A bounding volume hierarchy using axis-aligned bounding boxes is the recommended approach, though other structures such as a kd-tree or uniform grid are acceptable with justification.

- *Best-case demonstration*: Design a scene containing a large number of primitives, at minimum several thousand, and measure render time with and without the acceleration structure. Report the speedup as a ratio. Explain why this scene benefits strongly from the structure, for example because rays quickly cull large portions of the scene.
- *Worst-case demonstration*: Design a scene that obtains limited benefit from the acceleration structure and explain why. Consider scenes where primitives are poorly distributed spatially, where the hierarchy degenerates, or where the overhead of tree traversal approaches or exceeds the cost of brute-force intersection.

Provide timing measurements for both cases, reported clearly, for example as a table with render resolution, primitive count, time without acceleration, time with acceleration, and speedup ratio.

Output images acc_best, acc_worst, acc_complex. The last should show a complex scene with hundreds of geometry and multiple lights and report computational savings.

2.3.2 Test Suite

- Construct a structured test suite covering the following three categories. For each test, render the scene with your renderer and with PBRT-v4 and compare the outputs as described.
- *Camera and intersections*: Verify that the boundaries of rendered objects align correctly at the pixel level with those produced by PBRT for the same scene and camera configuration. Verify that the field-of-view parameter produces the same framing as PBRT. Describe your comparison methodology, for example pixel-difference images or boundary detection.
- *Shading*: Select a BRDF such as Phong or Blinn-Phong and measure the quantitative error between your DRT shading estimates and PBRT's output at a small set of specific pixels. Report the error numerically, for example as absolute difference per channel or relative error. Discuss possible sources of discrepancy including sampling noise, approximations, and tone mapping differences.

- *Error handling*: Design at least ten scene files that contain deliberate errors, for example an unknown primitive type, a malformed transform, a missing texture file, or an out-of-range parameter. Show the corresponding log file output from your renderer and confirm that each error is detected, described clearly, and handled gracefully, meaning the renderer does not crash silently.

The test suite must be self-contained and executable by other students. Each test should include a scene file, a reference image (rendered using pbrt-v4) and a reference log file. Write a script or instructions for running comparisons. I.e. to check if the renderer passes or fails the test based on its output.

Output images testsuite_ref_n, testsuite_my_n where n is the scene number. Design test scenes (.pbrt) testscene_n. In the report tabulate scens that fall under each category of tests. Report the quantitative errors across each category (compared to ref images) and also the percentage of successes in the error handling category.

2.4 Final Phase

2.4.1 Importance Sampling

Extend the DRT to support importance sampling of the material (BRDF). Rather than sampling the hemisphere uniformly for indirect illumination, draw samples distributed proportional to the BRDF or a suitable approximation of it. For example, a Cook-Torrance BRDF model.

Design at least two scenes with non-trivial geometry and BRDFs and measure the variance of the DRT estimator at specific pixels as a function of sample count, both with and without importance sampling. Present results as plots---for example variance versus sample count on a log-log scale. Discuss the reduction in variance and show example comparisons of when importance sampling is effective.

Output images is_unif1, is_brdf1, is_unif2, is_brdf2. Point to regions where differences are visible using arrows or zoomed insets.

2.4.2 Pooled Test Suite Results

A set of test cases will be provided by the course organiser, supplemented by cases submitted by students including your own student test suite from Module 3. Run the complete pooled test suite against your final renderer and include a results summary in the final report. The summary should report the total number of test cases run, the number of tests passed and failed broken down by category, and a discussion of any failures including likely causes and whether they reflect fundamental limitations of your implementation.

Output tabulated results for each of the categories. Examples of failure cases or error handling that was missed, with explanation.

3. REPORT STRUCTURE

3.1 General Guidance on Writing Reports

Reports must be concise, precise, and structured. Convey information about your implementation, design choices, and the results you are presenting. For each task or feature within a module report, include the following:

- *Key design decisions*: Describe the class structure, algorithmic choices, parameter values, and any assumptions made. Explain why each decision was made, not just what was decided.

- *Difficulties encountered*: Describe any implementation challenges, incorrect results, or failed test cases. Explain how you diagnosed the problem and what change resolved it. Do not omit failures; they are part of the assessment.
- *Test cases developed*: Describe the scene or input used to test the feature, what property of the feature the test exercises, and how you verified correctness.
- Example renders: Include rendered images with captions that direct the reader's attention to the specific aspect of the image that demonstrates the feature. Do not include images without explanation.
- Observations and insights: Note any non-obvious behaviour, unexpected performance characteristics, or parameter sensitivities you encountered.
- *Prompt development where applicable*: If you used a generative AI tool to assist in writing code, describe briefly how you formulated the prompt and how much manual revision was required. This is for transparency; there is no mark penalty or bonus for AI use per se.

3.2 Module Report Structure

Each of the three module reports must be structured with sections and subsections that mirror the tasks described in Section 2. The structure below is a template; you may add subsections where needed.

Module 1 report:

- WSRT with one subsection per feature covering specular reflection, refraction, shading, lights, shadows, textures, and geometry
- DRT with one subsection per feature covering meshes, textures, area lights, BRDF, depth of field, and tone mapping
- PBRT scene compatibility with notes on parsing and any unsupported features
- Profiling results with baseline timing measurements

Module 2 report:

- Camera covering orthographic vs perspective and field of view
- Antialiasing covering aliasing demonstration, supersampling, and performance discussion
- Sampler comparison covering methodology, renders, quantitative results, and discussion
- DRT parameter study covering scene designs, renders, and analysis

Module 3 report:

- Acceleration data structure covering design, best-case and worst-case results, and timings
- Test suite covering description of all test cases, comparison methodology, and results

3.3 Final Report Structure

The final report consolidated as exam_number.pdf consolidates all work. The following structure is suggested:

- Design Considerations: A brief section of one to two pages describing overall architectural decisions; how the codebase is organised, how scene files are parsed, how the two integrators are structured, and any cross-cutting design choices such as representation of rays, materials, and intersection results.
- Use of Generative AI: Describe the extent of prompting, curating and fixing (debugging) for generating working code in each module/section below. Also describe whether and how Language Models were used in preparing your report.
- One section for each Module (1, 2, 3)
    - Report at checkpoint
    - Timeliness bonus (justification/explanation of changes if any)

- Final Touches: Describe your implementation of importance sampling along with design, methodology, quantitative results including variance plots, and discussion.
- Test-Suite Results: Summary table and discussion of pooled test suite results.
- Exceptionalism: If you would like to submit features beyond the specs, see Section 5.
- Conclusion: A short section of approximately half a page summarising what was implemented, what worked well, what did not, and what you would improve or extend given more time.

4. Marking scheme

Module 1	**15**
Whitted style raytracer	6
Distributed raytracer	9
Module 2	**24**
Camera	4
Antialiasing	4
Sampler	8
Parameters	8
Module 3	**16**
Acceleration structures	6
Student's Test suite dev	10
Advanced	**10**
Importance sampling	5
Class Test Suite output	5
Timeliness Bonus	**15**
Module 1	5
Module 2	5
Module 3	5
Exceptionalism	**20**
TOTAL	**100**

Each graded feature, say antialiasing, will be assessed across the following weighted dimensions. A failure to clarify the following in the report will lead to a reduction of marks even if the feature is implemented and performs well.

50%	Output images, tables	As per specifications within each subsection.
50%	Explanations	Justification of design, test scene and explanation of output, with specific highlighting (arrows or insets).

5. Exceptionalism

20 marks out of the 100 for the coursework, are reserved for demonstration of exceptionalism, or development of features that are non-trivially beyond specifications. Since the use of AI coding assistants is permitted, there is an element of subjectivity on what constitutes exceptionalism. For example, although an extension of DRT to path tracing is non-trivial, it is quite likely that ELM will generate all the code necessary via a simple prompt. Thus, such an extension will not receive marks for exceptionalism. Typically, students who receive high marks for this section would develop some elements from the lectures, which are not already in the specifications, into the coursework and present a detailed analysis of that feature with examples. For instance, the feature might involve measuring and comparing numerical integration errors in path-tracing using the theory that will be taught in a lecture on the analysis of errors in numerical integration. In this case, different sampling strategies may be compared at different pixels, to arrive at conclusions on the pros and cons of strategies under different circumstances. If you are planning to aim for marks on exceptionalism, it is strongly recommended that you contact the instructor between Weeks 4 and 6 to plan your work.

6. Getting started

To help you get started with module 1 and to get in the flow of using AI coding assistants, this section recommends a set of steps. In case you face difficulties in any of these steps, please approach the course organiser and/or tutors at the lab sessions as early as possible.

- Read Sections 1 and 4 in detail
- Read 2.1
- A first C++ program
    - Familiarise yourself with compilation and linking procedures, particularly for C++.
    - Install a C++ compiler
    - Choose your editor or Integrated Development Environment (IDE) for the project
    - Prompt your favourite model (ELM with Claude Sonnet 4.6 is recommended) to generate a C++ program main.cpp to display “Hello world”.
    - Compile main.cpp and build an executable (binary) in the output directory.
    - Run the executable and verify that it outputs “Hello World”
- Reading PBRT files
    - Prompt the AI to generate code for reading PBRT files. Sometimes it can help to provide a link to the specification online.
    - Organise that code within your source directory as appropriate. For example, as a function in sceneReader.cpp
    - Use a prompt to generate test cases for the reader.
    - Append this with scenes of your own or they can be downloaded---don’t forget to acknowledge them if you obtained them from online.
    - Run the test cases to verify that the reader works as expected.
    - Write a log file with success/failure messages
    - Introduce known errors in a few scenes and test whether they are caught correctly and reported in the log
- Simple raytracer test
    - Use a prompt to generate a minimal raytracer that
        - Has all the code in a single .cpp file
        - only considers spherical objects
        - shades as binary. i.e. if an intersection with a sphere is found, shade that pixel 1 otherwise the pixel is 0.
        - Uses an orthographic camera

        - Writes out a .ppm image
    - Write a simple PBRT file with a single sphere
    - Build (compile and link) this raytracer and test if the image is correctly rendered
    - Go through the code for this raytracer and compare it to the lecture slides
    - Note down the components you will need for your WSRT and how they will interact
- Now you are ready to develop your WSRT by adding components in separate .cpp files
    - General utility (vector, ray, image)
    - Core raytracing (camera, scene, objects, lights)
    - Whitted-style raytracing using the above
    - Keep the shader simple (binary)
    - As you add elements, test your renderer incrementally
    - Use a simple scene with a single object (triangle, sphere or plane) to test
    - Add Blinn-Phong shading as outlined in class